IoT Temperature & Humidity Monitoring System

Design Decisions & Rationale

Nikolas Novachuk

Intro Engineering • Spring 2026

A full-stack IoT platform for collecting outdoor temperature and humidity from Arduino sensor nodes, validating accuracy against official weather, running statistical analysis, and exploring data through AI chat.

https://github.com/nikholasnova/TH-Dashboard

Contents

How It Started

This project started with a simple brief: build an Arduino temperature and humidity sensor using the provided kit and collect some data. How to collect the data and what to do with it was left open-ended. The instructor mentioned it might be cool if someone figured out how to display it on the web. So I began planning a system that collects temperature and humidity data from physical sensors and displays it on a web dashboard.

The twist was making it genuinely useful and technically interesting without spending any money. Every service used in this project runs on a free tier: Supabase for the database, Vercel for web hosting, WeatherAPI.com for weather reference data, Google Gemini for the AI chat, and Resend for email alerts.

The target audience is an intro engineering class. That means the system needs to be reproducible, another student should be able to fork the repo, follow the setup guide, and have a working system within an afternoon even without deep technical knowledge. This ruled out anything that requires specialized infrastructure, paid accounts, or deep DevOps knowledge. At the same time, I wanted the project to demonstrate real engineering patterns: data pipelines, authentication, monitoring, statistical analysis, and AI integration. The balance between a production-grade system and a simple system drove most of the decisions documented here.

The project is a public GitHub repository. Every design choice had to account for the fact that credentials cannot be committed, setup instructions need to be complete, and the code needs to be readable by someone who did not write it.

The scope grew incrementally. The initial version had two hardcoded sensors posting to Supabase and a basic dashboard with live readings. Over time, features were added in response to real needs: deployments because we wanted to move sensors between locations, weather comparison because we wanted to validate sensor accuracy, N-node support because we added more hardware, analysis because the project goal at the end was to write a paper based on your data, and AI chat because it made exploring the data significantly faster.

The end-to-end data path is: DHT20 sensor reads temperature and humidity via I^2C, the Arduino accumulates and averages readings, POSTs a JSON payload over HTTPS to Supabase's REST API, Supabase stores the row in a Postgres table with server-side timestamps, and the Next.js dashboard polls the database every 30 seconds to update the UI. Weather data follows a parallel path: a Vercel cron triggers an API route that fetches conditions from WeatherAPI.com and inserts them into the same readings table with a different `source` value. Both paths converge in the same database, which means every query, chart, and AI tool works identically for sensor and weather data.

Choosing the Foundation

The Database Problem

The first big decision was how to get data from an Arduino into a database. I needed something the Arduino could write to directly over HTTPS without an intermediary server. Firebase was the obvious choice, but it would have required either a Firebase SDK (which does not exist for Arduino) or a custom Cloud Function acting as a relay. A custom backend would mean maintaining a server, which adds cost and complexity that I was trying to avoid.

Supabase solved this. It exposes a REST API backed by Postgres, which means the Arduino can POST a JSON payload to `/rest/v1/readings` with just an anon key in the request headers. No SDK, no WebSocket, no authentication flow. The Arduino firmware constructs a raw HTTP request string and sends it over `WiFiSSLClient`. One less obvious benefit is that Supabase handles TLS termination. The Arduino connects to port 443 and the R4 WiFi's built-in TLS stack handles certificate validation. I did not need to manage certificates, pin roots, or deal with TLS handshake errors.

But Supabase gave me much more than just a REST endpoint. Postgres features turned out to be genuinely powerful: RPC functions for pushing aggregation queries to the database, Row Level Security for fine-grained access control, exclusion constraints for preventing overlapping deployment time windows, and partial unique indexes for weather deduplication. The schema file ended up at 658 lines defining 5 tables, 8 RPC functions, 3 constraint checks, and multiple index strategies. All of this runs on the database side, which means the web app stays thin.

The free tier provides 500MB of storage. I estimated that two sensors running for a full semester would produce roughly 115,000 rows, well under the limit. They also do not charge for API calls.

The built-in Auth system was another factor. I needed a login page to protect the dashboard, and Supabase Auth integrates directly with RLS policies. The `anon` role can INSERT readings (for Arduino), while the `authenticated` role can SELECT readings and execute RPC functions (for the dashboard). This is simpler than spending hours hardening the chat and database paths from potential bad actors. The `service_role` key is used only on the server side and bypasses RLS entirely.

One Project, Not Two

The dashboard needs both client-rendered pages (charts, comparisons, analysis) and server-side API routes (weather cron, keepalive, AI chat). I considered a plain React SPA with a separate Express backend, but that would mean two deployments, two sets of environment variables, and CORS configuration. Next.js App Router handles both in one framework, and Vercel provides free hosting with built-in cron support. Deploying is a git push.

The root layout wraps the app in a specific provider chain: `ThemeProvider`, then `AuthProvider`, then `DevicesProvider`, then `ChatPageContextProvider`. This makes theme, auth, the device list, and chat context available on every page without prop drilling. Getting this order wrong produces subtle bugs. For example, if `DevicesProvider` renders outside `AuthProvider`, the Supabase client will not have a session token and the devices query will fail silently with an empty result, making the dashboard show "No devices configured" even when devices exist.

The Hardware

The Arduino Uno R4 WiFi was chosen because it is an official Arduino board with built-in WiFi. Students already familiar with Arduino IDE can use it without learning a new platform. The DHT20 sensor communicates over I²C (address `0x38`), which keeps wiring simple, only four wires. The optional 16x2 LCD uses parallel 4-bit wiring, which is more complex but is what the class kit provided.

I intentionally kept the firmware as a single `.ino` file. For this class, being able to scroll through one file and see everything from WiFi connection to sensor reading to HTTP upload is more valuable than clean separation of concerns. Credentials live in a `secrets.h` file that is `.gitignore`'d, with a template in the repo.

One constraint worth noting: the board only supports 2.4GHz WiFi. Many campus and modern home networks default to 5GHz, and the board will fail silently on those. This ended up being the most common issue reported by other students trying to replicate the setup.

The Data Pipeline

Why Average Before Uploading

The sensor reads temperature and humidity every 15 seconds, but only uploads an average every 3 minutes. Each upload represents the average of approximately 12 readings. The primary motivation is the Supabase free tier. Uploading every 15 seconds would produce 5,760 rows per day per sensor. With 3-minute averaging, that drops to 480, a 12× reduction. Over a 120-day semester with two sensors, that is 115,200 rows instead of 1.38 million.

Beyond the row count, averaged data smooths out sensor noise. The DHT20 has a specified accuracy of ±0.5°C, and individual reads jitter within that range. A 3-minute average gives a more representative picture of actual conditions, and cleaner input data produces more meaningful results in the statistical analysis.

The database stores Celsius. Fahrenheit conversion happens only in the UI and on the LCD display. Storing Celsius is a deliberate choice. It is the standard scientific unit, and converting for display is a trivial `C × 9/5 + 32` that can be done anywhere.

In hindsight, 3 minutes is still a bit redundant. I have found the DHT20 sensors to last about 120,000 reads before they start degrading, and I observed temperature changing meaningfully every 5 or so minutes. I would most likely use a 5-minute interval going forward.

When Uploads Fail

When an upload fails (connection refused, timeout after 10 seconds, or non-2xx response), the Arduino retains its buffer. The accumulated sums and count are not reset. It schedules a retry with linear backoff: 30 seconds after the first failure, 60 after the second, 90 after the third, capped at 3 minutes.

```
unsigned long backoff = min(30000UL * consecutiveFailures,
                            SEND_INTERVAL_MS);
lastSendTime = now - SEND_INTERVAL_MS + backoff;
```

This sets `lastSendTime` such that the main loop's timing check will fire after the desired backoff period, without needing a separate retry state machine.

I chose linear over exponential backoff because the retry window is already short and the failure mode is almost always a transient WiFi dropout that resolves in seconds. Exponential backoff makes sense for APIs with rate limiting where you want to back off aggressively, but not here.

The key design property is that **no data is lost during transient network issues**. The buffer keeps accumulating readings even while retrying. If the network is down for 9 minutes, the first successful send will contain the average of roughly 36 readings spanning that entire gap. The trade-off is that a longer outage produces a single data point covering a wider time window rather than multiple points at the normal 3-minute cadence, but this is preferable to losing the data entirely.

WiFi Drops

The original firmware assumed the WiFi connection would stay up after `setup()`, which led to silent upload failures. The send function would fail to connect but the buffer would be reset anyway. The fix was a WiFi status check at the top of every `loop()` iteration that calls `connectWiFi()` if the connection dropped. It attempts up to 20 reconnection tries with a spinning animation on the LCD (`|`, `/`, `-`, `\`) so you can see the state without a serial monitor. Together with the retry-with-backoff on upload failure, this makes the system resilient to WiFi dropouts lasting up to several minutes.

Growing Beyond Two Sensors

The project originally hardcoded two devices: `node1` and `node2`. When I needed to add a third sensor, I realized the hardcoded approach was very short-sighted. Device IDs were scattered across firmware constants, database seed data, dashboard components, keepalive monitoring, and weather integration. Refactoring to dynamic devices touched nearly every file in the project.

The solution was a `devices` table that stores the device ID, display name, color, active status, monitoring toggle, and sort order. Every part of the system that previously used a hardcoded device list now reads from this table. The dashboard knows which live cards to render, the keepalive route knows which devices to monitor, and the weather route knows which deployments to fetch conditions for.

I also added an auto-registration trigger on the `readings` table. When a new `device_id` appears in an INSERT, the trigger optionally creates a `devices` row, gated behind a feature flag. The trigger uses `SECURITY DEFINER` so it can insert into the `devices` table even when the original INSERT comes from the Arduino's `anon` role. This means adding a new sensor is: flash the sketch with a new `DEVICE_ID`, power it on, and the system discovers it automatically.

The dashboard grid adapts to the number of active devices: 1 device gets a centered column, 2 get a responsive 2-column grid, 3 use up to 3 columns, and 4+ go up to 4 columns. The dashboard looks good whether you have 1 sensor or 8 without any manual layout configuration.

Making Sense of Location

Deployments

A deployment represents a placement window: a specific sensor at a specific location during a specific time range. This concept exists because the same physical sensor might be placed in different locations over the course of the class. On a patio one week, at a friend's house the next. Without deployments, all readings from `node1` would be lumped together regardless of where the sensor was physically located. When I run a hypothesis test comparing two deployments, I am comparing two distinct physical placements, not two random time slices from mixed environments.

The database enforces integrity with an exclusion constraint using `btree_gist` that prevents overlapping time windows per device. A unique partial index prevents multiple active deployments per device. The `delete_deployment_cascade` RPC function deletes both the deployment and its associated readings in a single transaction, but only readings that do not also belong to another deployment's window.

The ZIP code field on deployments enables weather comparison. When a deployment has a ZIP code, the weather cron fetches official conditions for that location and the dashboard shows percent error. I chose ZIP codes because they are simpler to enter than coordinates and WeatherAPI.com accepts them directly.

Weather Validation

Every 15 minutes, a Vercel cron job fetches current conditions from WeatherAPI.com for each active deployment's ZIP code and inserts them as readings with `source = 'weather'` and `device_id = 'weather_<sensor_id>'`. I made a deliberate choice to store weather data in the same `readings` table as sensor data. The alternative was a separate `weather_readings` table, which would have required duplicating every RPC function, chart query, and stats computation. By using the same table with a `source` column and a device ID convention, all existing query infrastructure works with weather data without modification. The charts page can overlay sensor and weather lines on the same graph. The AI chat can compare a sensor to its weather counterpart in one tool call.

The Compare page computes percent error with color coding: green below 3%, amber between 3–5%, red above 5%. Each live card on the dashboard also shows this inline: "vs Official: 74.2°F (1.8% Error)", so you can see sensor accuracy at a glance.

Weather deduplication was a real concern because Vercel crons can occasionally retry or be triggered manually. I built a two-layer defense: the route code checks for existing rows in the current 15-minute UTC bucket before inserting, and a unique partial index in the database catches any race conditions. Duplicate inserts are counted as skips rather than failures. The route also deduplicates API calls by ZIP code. If two deployments share the same ZIP, the weather API is called once and the result is inserted for both weather device IDs.

Analysis Without a Server

The project goal at the end was to write a paper based on collected data, which meant I needed statistical analysis. The challenge was that Vercel does not natively run Python, and I did not want to add a separate Python backend just for stats. The solution was Pyodide, a WebAssembly port of CPython that runs numpy, pandas, scipy, and statsmodels entirely in the browser.

The data flow works like this: TypeScript fetches readings from Supabase, serializes them as JSON, passes them into the Python runtime, executes the analysis script, and gets results back as JSON. Six analysis types are available:

Analysis	Method
Descriptive statistics	Mean, median, std, skewness, kurtosis, standard error
Correlation	Pearson r, linear regression, scatter plots
Hypothesis testing	Welch's t-test with Cohen's d effect size
ANOVA	One-way with Tukey HSD post-hoc (3+ deployments)
Seasonal decomposition	Additive model, 24-hour period
Forecasting	Holt-Winters exponential smoothing, 24-hour seasonal

The hypothesis testing is particularly useful for the class paper. You can make a statistically rigorous claim like "the temperature difference between Location A and Location B was significant ($t = 3.42$, $p = 0.001$, $d = 0.85$)" and have the numbers to back it up.

The trade-off is a 10-second cold start the first time you visit the analysis page. Loading the WebAssembly runtime plus all four Python packages from a CDN takes time. I mitigate this with a singleton loader that caches the runtime across page navigations, so subsequent visits start instantly. I considered preloading Pyodide on the dashboard page so it would be ready when the user navigates to analysis, but decided against it because it would increase memory usage on every page and download megabytes of WASM that might never be needed. This cold start is the single biggest UX issue in the project, but the trade-off of a one-time wait felt better than a universal resource cost.

Edge cases required careful handling. Correlation analysis checks that both temperature and humidity have non-zero standard deviation before computing Pearson's r. Seasonal decomposition requires at least 2 full seasonal cycles. Forecasting trims partial days from the end of the series to avoid seasonal misalignment. All results are sanitized before JSON serialization to handle numpy types and non-finite values that would break `JSON.stringify`. These guards were added after encountering crashes with short or flat datasets during testing.

Teaching an AI to Read Sensor Data

The floating AI chat is available on every page. It sends messages to a server route that authenticates the user, checks rate limits (30 requests per 15 minutes), and forwards the conversation to Gemini 2.5 Flash with 7 read-only tools that give the AI access to all project data through the same Supabase queries the dashboard uses.

Tool	Purpose
`get_deployments`	List deployments with optional filters
`get_deployment_stats`	Aggregate stats per deployment
`get_readings`	Raw readings, sortable by temperature or humidity
`get_device_stats`	Stats per device (defaults to last 30 days)
`get_chart_data`	Time-bucketed averages for trends
`get_report_data`	All deployments with full statistics in one call
`get_weather`	Latest stored weather readings

Google Gemini was my choice because it offers a free API tier with function-calling support and streaming. The system prompt is pretty large. It includes instructions for handling common question patterns, report generation formatting, sensor-vs-weather comparison framing, and device ID conventions. It is dynamically augmented at request time with the current Arizona local time, registered device IDs, and known deployments. This gives the AI enough context to match natural language references like "the patio deployment" to actual database records. I would have liked to keep the system prompt leaner, but the Flash models are not the best at tool calling and have high failure rates without sufficient guidance.

One problem I ran into was the AI burning through too many tool calls for simple questions. A query like "compare node1 and node2" would trigger a cascade of lookups (find deployments for each device, get stats for each, fetch readings) when a single call to `get_device_stats` with no filter returns all devices at once. I added explicit efficiency rules to the system prompt and made `start` and `end` parameters optional (defaulting to the last 30 days) so the model does not waste a call looking up date ranges.

Another issue was large tool results overwhelming the model's context. When the AI fetched 2,000 raw readings, the JSON payload was massive and the model would loop endlessly trying to process it. I added a 30KB cap on tool result payloads. Large arrays get truncated with a note telling the model to use aggregate tools instead. When the tool loop exhausts without generating text, the model is now prompted to summarize whatever data it gathered rather than failing silently.

Streaming responses come through a `TransformStream` with status markers so the chat UI shows what the AI is doing. On the client side, a time-based word drip renders text at roughly 48 words per second, decoupled from network chunk sizes. Without this, text would arrive in bursts. A 50-word chunk from Gemini would appear all at once. The drip smooths it into a steady flow that feels like ChatGPT or Claude.

Knowing When Things Break

The keepalive route runs every 10 minutes via Vercel cron. For each monitored device, it fetches the latest reading and classifies it: `ok` if the reading is recent and within bounds, `missing` if no readings have ever been received, `stale` if the latest reading is older than the threshold (default 10 minutes), or `anomaly` if the sensor values are outside the DHT20's operating limits (-40 to 80°C, 0 to 100% humidity).

The deduplication logic is the critical part. A sensor that stays offline for days generates exactly one alert email, not one every 10 minutes. `shouldSendProblemAlert` returns true only when transitioning from `ok` to a problem state, or when the problem type changes. `shouldSendRecoveryAlert` fires only on transition back to `ok`. The `last_alert_sent_at` timestamp is recorded regardless of whether the email actually delivered, so a down email provider does not cause infinite retry attempts.

Emails go through Resend with subject lines like "[IoT Alert] node1 OFFLINE / STALE" and include the device ID, status, reason, last seen timestamp, and latest values. The architecture supports multiple notification channels through a `dispatchNotifications` function that runs all channels in parallel. Currently only email is implemented, but adding SMS or Slack would be a matter of adding another function to the array.

The Hard Parts

Row Level Security

Getting RLS right was one of the more frustrating parts of the project. The policies needed to serve three different access patterns simultaneously: Arduino devices (anon role) need INSERT on `readings`, authenticated dashboard users need SELECT and CRUD on various tables, and server-side routes need unrestricted access for weather inserts, keepalive state upserts, and cascade deletes.

The solution was careful per-table policies combined with RPC functions that use `SECURITY DEFINER` to run as the function owner. This lets authenticated users delete readings through the cascade RPC even though they lack direct DELETE permission on the `readings` table. The auto-registration trigger also uses `SECURITY DEFINER` since it needs to write to `devices` from an anon-initiated INSERT on `readings`.

Policies that look correct by themselves can interact in strange ways, and Supabase's error messages for RLS violations are not always descriptive. I ended up testing each combination manually. The schema file includes explicit `DROP POLICY IF EXISTS` before each `CREATE POLICY` to make migrations idempotent. This was necessary because the schema evolved over time and re-running it on a partially-applied database needed to work without errors.

Free Tier Constraints as Architecture

The Supabase free tier shaped the entire data pipeline. Three-minute averaging was not just a nice-to-have, it was necessary to keep row counts under control. Postgres RPC functions push aggregation to the database rather than fetching raw rows client-side. The `get_dashboard_live` function uses three `UNION ALL` branches to return latest readings, weather data, and sparkline data for all devices in a single query. Before this function existed, the dashboard made 3 queries per device. With N-node support, that would have scaled to $3N$ queries per poll cycle. The batched RPC reduces it to 1 regardless of device count.

A module-level cache stores the last known dashboard state and initializes the component on re-mount during client-side navigation, preventing a flash of empty state when navigating away and back.

Making Everything Idempotent

Vercel's cron execution on the free tier is best-effort. Jobs can be delayed, skipped, or retried. This meant both the weather and keepalive routes had to be idempotent. For weather, the two-layer deduplication handles retries gracefully. For keepalive, the state-transition-based alerting means a delayed cron just extends the detection window slightly rather than producing duplicate alerts.

What I Would Do Differently

If this were not a school project constrained to free tiers, the architecture would look different. I would use an MQTT broker between the sensors and the database. Persistent TCP connections instead of per-request TLS handshakes, QoS levels for guaranteed delivery, and decoupling sensors from the database so switching backends would not require reflashing firmware. The dashboard would use WebSocket push instead of 30-second polling. The Python analysis would run server-side on Lambda with native numpy and scipy, eliminating the 10-second Pyodide cold start. Scheduled tasks would use a dedicated scheduler like AWS EventBridge instead of Vercel's best-effort crons. And the auth model would support multiple users with role-based access.

All of these improvements add complexity and cost, which were the two things I most wanted to avoid. For a school project meant to be forked and reproduced by other students, the simpler architecture is the correct choice.

What I Learned

The project grew from a barebones two-sensor dashboard into a comprehensive IoT platform with weather validation, statistical analysis, and AI chat. Each feature was added to solve a real problem encountered during use.

The most important lesson was that starting simple and iterating is always better than trying to build everything at once. But that does not mean ignoring scalability. I now think about how I would scale something from the very beginning, even if the first implementation is intentionally simple.

The second lesson was that free-tier constraints are net positives. The 3-minute averaging, adaptive chart bucketing, batched RPC queries, and weather deduplication all exist because of free-tier limits. These constraints forced me to think about efficiency in ways that a pay-as-you-go setup would not have.

Thank you for reading.

13,611 lines of source code • 328 tests • 72% statement coverage

`https://github.com/nikholasnova/TH-Dashboard`